

Convertir notación de infijo a prefijo

julio 5, 2022 Rudeus Greyrat

Mientras usamos expresiones infijas en nuestra vida cotidiana. Las computadoras tienen problemas para entender este formato porque deben tener en cuenta las reglas de precedencia de los operadores y también los corchetes. Las expresiones de prefijo y sufijo son más fáciles de entender y evaluar para una computadora.

Dados dos operandos a y b un operador $\odot$, la notación infija implica que $\odot$ se colocará entre a y b , es decir $a \odot b$. Cuando el operador se coloca después de ambos operandos, es decir $ab\odot$, se llama notación de sufijo. Y cuando el operador se coloca antes de los operandos, es decir $\odot ab$, la expresión en notación de prefijo.

Dada cualquier expresión infija, podemos obtener el formato de prefijo y sufijo equivalente.

Ejemplos:

Input : $A * B + C / D$

Output : $+ * A B / C D$

Input : $(A - B/C) * (A/K-L)$

Output : $* - A / B C - / A K L$

Para convertir una expresión de infijo a sufijo, consulte este artículo [Stack | Juego 2 \(Infijo a Postfijo\)](#) . Usamos lo mismo para convertir Infix a Prefix.

- Paso 1: Invierta la expresión infija, es decir, $A+B*C$ se convertirá en $C*B+A$. Tenga en cuenta que al invertir cada '(' se convertirá en ')' y cada ')' se convertirá en '('.
- Paso 2: Obtenga la expresión del sufijo «casi» de la expresión modificada, es decir, $CB*A+$.
- Paso 3: Invierta la expresión del sufijo. Por lo tanto, en nuestro ejemplo, el prefijo es $+A*BC$.

Tenga en cuenta que para el Paso 2, no usamos el algoritmo de sufijo tal como está. Hay un cambio menor en el algoritmo.

Según <https://www.geeksforgeeks.org/stack-set-2-infix-to-postfix/> , tenemos que sacar todos los operadores de la pila que tengan una precedencia **mayor o igual** que la del operador escaneado . Pero aquí, tenemos que extraer todos los operadores de la pila que tengan una precedencia **mayor** que la del operador escaneado. Solo en el caso del operador «^», extraemos operadores de la pila que tienen una precedencia **mayor o igual** .

A continuación se muestra la implementación en C++ del algoritmo.

C++

```
// CPP program to convert infix to prefix
#include <bits/stdc++.h>
using namespace std;

bool isOperator(char c)
{
    return (!isalpha(c) && !isdigit(c));
}

int getPriority(char C)
{
    if (C == '-' || C == '+')
        return 1;
    else if (C == '*' || C == '/')
        return 2;
    else if (C == '^')
        return 3;
    return 0;
}

string infixToPostfix(string infix)
{
    infix = '(' + infix + ')';
    int l = infix.size();
    stack<char> char_stack;
    string output;
```

```
for (int i = 0; i < l; i++) {

    // If the scanned character is an
    // operand, add it to output.
    if (isalpha(infix[i]) || isdigit(infix[i]))
        output += infix[i];

    // If the scanned character is an
    // '(', push it to the stack.
    else if (infix[i] == '(')
        char_stack.push('(');

    // If the scanned character is an
    // ')', pop and output from the stack
    // until an '(' is encountered.
    else if (infix[i] == ')') {
        while (char_stack.top() != '(') {
            output += char_stack.top();
            char_stack.pop();
        }

        // Remove '(' from the stack
        char_stack.pop();
    }

    // Operator found
    else
    {
        if (isOperator(char_stack.top()))
        {
            if(infix[i] == '^')
            {
```

```
        while (getPriority(infix[i]) <= getPriority(char_stack.top()))
        {
            output += char_stack.top();
            char_stack.pop();
        }

    }
    else
    {
        while (getPriority(infix[i]) < getPriority(char_stack.top()))
        {
            output += char_stack.top();
            char_stack.pop();
        }

    }

    // Push current Operator on stack
    char_stack.push(infix[i]);
}

}

while(!char_stack.empty()){
    output += char_stack.top();
    char_stack.pop();
}

return output;
}

string infixToPrefix(string infix)
{
    /* Reverse String
```

```
* Replace ( with ) and vice versa
* Get Postfix
* Reverse Postfix * */
int l = infix.size();

// Reverse infix
reverse(infix.begin(), infix.end());

// Replace ( with ) and vice versa
for (int i = 0; i < l; i++) {

    if (infix[i] == '(') {
        infix[i] = ')';
    }
    else if (infix[i] == ')') {
        infix[i] = '(';
    }
}

string prefix = infixToPostfix(infix);

// Reverse postfix
reverse(prefix.begin(), prefix.end());

return prefix;
}

// Driver code
int main()
{
    string s = ("x+y*z/w+u");
    cout << infixToPrefix(s) << std::endl;
```

```
    return 0;  
}
```

Java

```
// JAVA program to convert infix to prefix  
import java.util.*;  
  
class GFG  
{  
    static boolean isalpha(char c)  
    {  
        if (c >= 'a' && c <= 'z' || c >= 'A' && c <= 'Z')  
        {  
            return true;  
        }  
        return false;  
    }  
  
    static boolean isdigit(char c)  
    {  
        if (c >= '0' && c <= '9')  
        {  
            return true;  
        }  
        return false;  
    }  
  
    static boolean isOperator(char c)
```

```
{
    return (!isalpha(c) && !isdigit(c));
}

static int getPriority(char C)
{
    if (C == '-' || C == '+')
        return 1;
    else if (C == '*' || C == '/')
        return 2;
    else if (C == '^')
        return 3;

    return 0;
}

// Reverse the letters of the word
static String reverse(char str[], int start, int end)
{
    // Temporary variable to store character
    char temp;
    while (start < end)
    {
        // Swapping the first and last character
        temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        start++;
        end--;
    }
}
```

```
        return String.valueOf(str);
    }

    static String infixToPostfix(char[] infix1)
    {
        System.out.println(infix1);
        String infix = '(' + String.valueOf(infix1) + ')';

        int l = infix.length();
        Stack<Character> char_stack = new Stack<>();
        String output="";

        for (int i = 0; i < l; i++)
        {

            // If the scanned character is an
            // operand, add it to output.
            if (isalpha(infix.charAt(i)) || isdigit(infix.charAt(i)))
                output += infix.charAt(i);

            // If the scanned character is an
            // '(', push it to the stack.
            else if (infix.charAt(i) == '(')
                char_stack.add('(');

            // If the scanned character is an
            // ')', pop and output from the stack
            // until an '(' is encountered.
            else if (infix.charAt(i) == ')')
            {
                while (char_stack.peek() != '(')
                {
```



```
        output += char_stack.peek();
        char_stack.pop();
    }

    // Remove '(' from the stack
    char_stack.pop();
}

// Operator found
else {
    if (isOperator(char_stack.peek()))
    {
        while ((getPriority(infix.charAt(i)) <
                        getPriority(char_stack.peek()))
                || (getPriority(infix.charAt(i)) <=
                        getPriority(char_stack.peek())
                        && infix.charAt(i) == '^'))
        {
            output += char_stack.peek();
            char_stack.pop();
        }

        // Push current Operator on stack
        char_stack.add(infix.charAt(i));
    }
}

while(!char_stack.empty()){
    output += char_stack.pop();
}

return output;
}
```

```
static String infixToPrefix(char[] infix)
{
    /*
        * Reverse String Replace ( with ) and vice versa Get Postfix Reverse Postfix *
        */
    int l = infix.length;

    // Reverse infix
    String infix1 = reverse(infix, 0, l - 1);
    infix = infix1.toCharArray();

    // Replace ( with ) and vice versa
    for (int i = 0; i < l; i++)
    {

        if (infix[i] == '(')
        {
            infix[i] = ')';
i++;
        }
        else if (infix[i] == ')')
        {
            infix[i] = '(';
i++;
        }
    }

    String prefix = infixToPostfix(infix);

    // Reverse postfix
    prefix = reverse(prefix.toCharArray(), 0, l-1);
}
```

```
        return prefix;
    }

    // Driver code
    public static void main(String[] args)
    {
        String s = ("x+y*z/w+u");
        System.out.print(infixToPrefix(s.toCharArray()));
    }
}

// This code is contributed by Rajput-Ji
```

C#

```
// C# program to convert infix to prefix
using System;
using System.Collections.Generic;

public class GFG {
    static bool isalpha(char c) {
        if (c >= 'a' && c <= 'z' || c >= 'A' && c <= 'Z') {
            return true;
        }
        return false;
    }

    static bool isdigit(char c) {
```

```
    if (c >= '0' && c <= '9') {
        return true;
    }
    return false;
}

static bool isOperator(char c) {
    return (!isalpha(c) && !isdigit(c));
}

static int getPriority(char C) {
    if (C == '-' || C == '+')
        return 1;
    else if (C == '*' || C == '/')
        return 2;
    else if (C == '^')
        return 3;

    return 0;
}

// Reverse the letters of the word
static String reverse(char []str, int start, int end) {

    // Temporary variable to store character
    char temp;
    while (start < end) {

        // Swapping the first and last character
        temp = str[start];
        str[start] = str[end];
        str[end] = temp;
    }
}
```

```
        start++;
        end--;
    }
    return String.Join("",str);
}

static String infixToPostfix(char[] infix1) {
    String infix = '(' + String.Join("",infix1) + ')';

    int l = infix.Length;
    Stack<char> char_stack = new Stack<char>();
    String output = "";

    for (int i = 0; i < l; i++) {

        // If the scanned character is an
        // operand, add it to output.
        if (isalpha(infix[i]) || isdigit(infix[i]))
            output += infix[i];

        // If the scanned character is an
        // '(', push it to the stack.
        else if (infix[i] == '(')
            char_stack.Push('(');

        // If the scanned character is an
        // ')', pop and output from the stack
        // until an '(' is encountered.
        else if (infix[i] == ')') {
            while (char_stack.Peek() != '(') {
                output += char_stack.Peek();
                char_stack.Pop();
            }
        }
    }
}
```

```
    }

    // Remove '(' from the stack
    char_stack.Pop();
}

// Operator found
else {
    if (isOperator(char_stack.Peek())) {
        while ((getPriority(infix[i]) < getPriority(char_stack.Peek()))
            || (getPriority(infix[i]) <= getPriority(char_stack.Peek())
                && infix[i] == '^')) {
            output += char_stack.Peek();
            char_stack.Pop();
        }

        // Push current Operator on stack
        char_stack.Push(infix[i]);
    }
}

while (char_stack.Count!=0) {
    output += char_stack.Pop();
}

return output;
}

static String infixToPrefix(char[] infix) {
    /*
        * Reverse String Replace ( with ) and
        // vice versa Get Postfix Reverse Postfix *
    */
}
```

```
int l = infix.Length;

// Reverse infix
String infix1 = reverse(infix, 0, l - 1);
infix = infix1.ToCharArray();

// Replace ( with ) and vice versa
for (int i = 0; i < l; i++) {

    if (infix[i] == '(') {
        infix[i] = ')';
        i++;
    } else if (infix[i] == ')') {
        infix[i] = '(';
        i++;
    }
}

String prefix = infixToPostfix(infix);

// Reverse postfix
prefix = reverse(prefix.ToCharArray(), 0, l - 1);

return prefix;
}

// Driver code
public static void Main(String[] args) {
    String s = ("x+y*z/w+u");
    Console.Write(infixToPrefix(s.ToCharArray()));
}
}
```

 **prefix.Length()**

```
// This code is contributed by gauravrajput1
```

Producción

```
++x/*yzwu
```

Complejidad de tiempo:

las operaciones de pila como push() y pop() se realizan en tiempo constante. Dado que escaneamos todos los caracteres en la expresión una vez, la complejidad es lineal en el tiempo, es decir $\mathcal{O}(n)$.

Publicación traducida automáticamente

Artículo escrito por [Sayan Mahapatra](#) y traducido por Barcelona Geeks. The original can be accessed [here](#). Licence: [CCBY-SA](#)

expression-evaluation Stack Strings